C++ STL Algorithms

Interview Questions & Answers

60 Expert Questions — Sorting, Searching, Numeric, Parallel & Ranges

With Working Code Examples | C++11 through C++20

SECTION 1 — Sorting & Ordering Algorithms

Q1. What algorithm does std::sort use internally, and what is its complexity?

Answer

std::sort uses Introsort — a hybrid of Quicksort (average O(n log n)), Heapsort (worst-case fallback when recursion depth exceeds 2*log n), and Insertion Sort (for small sub-ranges, typically < 16 elements). This gives guaranteed O(n log n) worst-case with excellent average-case constants. It is NOT stable.

Example

```
#include <algorithm>
#include <vector>

std::vector<int> v = {5, 2, 8, 1, 9, 3, 7, 4, 6};

std::sort(v.begin(), v.end());          // ascending
std::sort(v.begin(), v.end(),
          std::greater<int>());          // descending

// Custom comparator
std::sort(v.begin(), v.end(),
    [](int a, int b){ return std::abs(a) < std::abs(b); });
```

Q2. When should you use std::stable_sort over std::sort?

Answer

Use stable_sort when equal elements must retain their relative original order. For example, sorting a list of employees by salary while keeping alphabetical order among same-salary employees. stable_sort uses merge sort: O(n log n) with O(n) extra memory, or O(n log² n) without. The stability guarantee makes it slower than std::sort.

Example

```
struct Employee { std::string name; int salary; };

std::vector<Employee> employees = {
    {"Alice",50000}, {"Bob",60000},
    {"Carol",50000}, {"Dave",60000}
};

// stable_sort: Alice before Carol (same salary, original order kept)
std::stable_sort(employees.begin(), employees.end(),
    [](auto& a, auto& b){ return a.salary < b.salary; });
// Result: Alice(50k), Carol(50k), Bob(60k), Dave(60k)
```

Q3. What does std::partial_sort do and when is it more efficient than std::sort?

Answer

std::partial_sort(first, middle, last) places the smallest (middle-first) elements in sorted order in [first, middle), leaving the rest in unspecified order. Complexity: O(n log k) where k = middle-first. It is more efficient than sorting the whole range when you only need the top-k elements (k << n).

Example

```
std::vector<int> v = {50,30,80,10,90,20,70,40,60};

// Get the 3 smallest elements, sorted
```

```
std::partial_sort(v.begin(), v.begin()+3, v.end());
// v[0..2] = {10, 20, 30} — rest is unspecified

// Top 3 largest (with comparator)
std::partial_sort(v.begin(), v.begin()+3, v.end(),
                  std::greater<int>());
// v[0..2] = {90, 80, 70}
```

Q4. Explain std::nth_element — what guarantee does it provide and at what complexity?

Answer

std::nth_element rearranges a range so that: the element at position nth is exactly the element that would be there if the range were fully sorted; all elements before nth are <= *nth; all elements after nth are >= *nth. Groups before/after are NOT sorted. Uses introselect: O(n) average, O(n) worst-case (not O(n²) like naive quickselect).

Example

```
std::vector<int> v = {5,2,8,1,9,3,7,4,6};

// Find the median (element that would be at index 4)
auto mid = v.begin() + v.size()/2;
std::nth_element(v.begin(), mid, v.end());
std::cout << "Median: " << *mid; // 5

// All elements before mid <= 5, all after >= 5
// (but neither group is sorted)

// Top-k without sorting: nth_element is O(n), sort is O(n log n)
std::nth_element(v.begin(), v.begin()+3, v.end(),
                 std::greater<int>());
// First 3 elements are the 3 largest (unordered among themselves)
```

Q5. What is std::sort_heap and how does it relate to make_heap?

Answer

std::sort_heap converts a max-heap (satisfying heap property) into a sorted ascending range. It is essentially the second phase of Heapsort. Complexity: O(n log n). You must first call make_heap to establish the heap property, or use a vector that was managed with push_heap. After sort_heap the heap property is destroyed.

Example

```
std::vector<int> v = {3,1,4,1,5,9,2,6,5};

// Phase 1: build heap — O(n)
std::make_heap(v.begin(), v.end());
// v[0] is now max (9)

// Phase 2: sort the heap — O(n log n)
std::sort_heap(v.begin(), v.end());
// v is now fully sorted ascending
// Heap property is destroyed — v is no longer a heap
```

Q6. What is std::is_sorted and std::is_sorted_until?

Answer

is_sorted returns true if the range is sorted in non-descending order (or by a custom comparator). is_sorted_until returns an iterator to the first element that breaks the sorted order — if the entire range is sorted it returns end(). Both run in O(n) and are useful for assertions and optimization decisions.

Example

```
std::vector<int> v1 = {1,2,3,4,5};
std::vector<int> v2 = {1,2,5,3,4};

std::cout << std::is_sorted(v1.begin(), v1.end()); // 1
std::cout << std::is_sorted(v2.begin(), v2.end()); // 0

auto it = std::is_sorted_until(v2.begin(), v2.end());
// it points to 3 (first out-of-order element)
std::cout << std::distance(v2.begin(), it); // 3
std::cout << *it; // 3
```

SECTION 2 — Searching & Binary Search Algorithms

Q7. What is the difference between std::lower_bound, std::upper_bound, and std::equal_range?

Answer

All three perform binary search (O(log n)) on sorted ranges. lower_bound returns iterator to first element NOT LESS THAN value. upper_bound returns iterator to first element GREATER THAN value. equal_range returns a pair [lower, upper) enclosing all elements equal to value — equivalent to calling both in one pass.

Example

```
std::vector<int> v = {1,2,2,3,3,3,4,5};

auto lo = std::lower_bound(v.begin(), v.end(), 3); // -> first 3
auto hi = std::upper_bound(v.begin(), v.end(), 3); // -> 4

// Count occurrences: O(log n)
std::cout << std::distance(lo, hi); // 3

// equal_range returns both at once
auto [beg, end] = std::equal_range(v.begin(), v.end(), 3);
std::cout << std::distance(beg, end); // 3

// Test membership on sorted range
bool found = lo != v.end() && *lo == 3; // true
```

Q8. What does std::binary_search return and what does it NOT tell you?

Answer

std::binary_search returns a bool — true if the value exists. It does NOT return an iterator to the element. If you need the position, use lower_bound. binary_search requires a sorted range. It is equivalent to checking lo != end && *lo == value where lo = lower_bound(...).

Example

```
std::vector<int> v = {1,3,5,7,9,11};

bool found = std::binary_search(v.begin(), v.end(), 7);
```

```
std::cout << found; // 1 (true)

bool missing = std::binary_search(v.begin(), v.end(), 4);
std::cout << missing; // 0 (false)

// To get position, use lower_bound:
auto it = std::lower_bound(v.begin(), v.end(), 7);
if (it != v.end() && *it == 7)
    std::cout << "Index: " << std::distance(v.begin(),it); // 3
```

Q9. How do std::find, std::find_if, and std::find_if_not differ?

Answer

std::find performs a linear O(n) search for a specific value by equality. std::find_if searches for the first element satisfying a unary predicate. std::find_if_not finds the first element NOT satisfying the predicate. All return end() on failure. For sorted ranges prefer binary_search or lower_bound for O(log n).

Example

```
std::vector<int> v = {1,3,5,7,8,10,12};

// Find specific value
auto it1 = std::find(v.begin(), v.end(), 7);
std::cout << *it1; // 7

// Find first even
auto it2 = std::find_if(v.begin(), v.end(),
    [](int x){ return x%2 == 0; });
std::cout << *it2; // 8

// Find first non-odd
auto it3 = std::find_if_not(v.begin(), v.end(),
    [](int x){ return x%2 != 0; });
std::cout << *it3; // 8
```

Q10. What is std::search and how does the C++17 Searcher object improve it?

Answer

std::search finds the first occurrence of a subsequence [s_first, s_last) within [first, last). Naive version is O(n*m). C++17 adds Searcher objects: std::default_searcher, std::boyer_moore_searcher (O(n/m) average), and std::boyer_moore_horspool_searcher. Pass a Searcher as the third argument for sub-linear search on large texts.

Example

```
std::string text = "the quick brown fox jumps over the lazy dog";
std::string pat  = "fox";

// Naive O(n*m)
auto it = std::search(text.begin(), text.end(),
                      pat.begin(), pat.end());
std::cout << std::distance(text.begin(), it); // 16

// C++17 Boyer-Moore: better for long patterns
auto bm = std::boyer_moore_searcher(pat.begin(), pat.end());
auto it2 = std::search(text.begin(), text.end(), bm);
std::cout << *it2; // f
```

Q11. What does std::search_n do?

Answer

std::search_n(first, last, count, value) finds the first position where value appears count consecutive times. Returns end() if not found. An optional comparator can replace equality. Runs in O(n). Useful for finding runs of identical values.

Example

```
std::vector<int> v = {1,2,2,2,3,4,4,5};

// Find first run of three 2s
auto it = std::search_n(v.begin(), v.end(), 3, 2);
if (it != v.end())
    std::cout << "Found at index "
              << std::distance(v.begin(), it); // 1

// With comparator (find 3 consecutive evens)
auto it2 = std::search_n(v.begin(), v.end(), 2, 0,
    [](int a, int /*b*/){ return a%2==0; });
std::cout << std::distance(v.begin(), it2); // 1 (first 2,2)
```

Q12. What is std::find_end and how does it differ from std::search?

Answer

std::find_end finds the LAST occurrence of a sub-range within a range (like rfind for sequences). std::search finds the FIRST occurrence. Both return end() of the outer range if not found. find_end is O(n*m) but searches from the back conceptually.

Example

```
std::vector<int> v = {1,2,3,1,2,3,1,2,3};
std::vector<int> pat = {1,2,3};

// std::search: first occurrence
auto first = std::search(v.begin(), v.end(),
                         pat.begin(), pat.end());
std::cout << std::distance(v.begin(), first); // 0

// std::find_end: LAST occurrence
auto last = std::find_end(v.begin(), v.end(),
                          pat.begin(), pat.end());
std::cout << std::distance(v.begin(), last); // 6
```

Q13. What does std::find_first_of do?

Answer

std::find_first_of(first1, last1, first2, last2) searches the first range for the first element that matches ANY element in the second range. It is similar to std::string::find_first_of but generalized to any range and comparator. Complexity: O(n*m).

Example

```
std::vector<int> data = {1,5,3,8,2,7,4};
std::vector<int> targets = {8, 2};

// Find first element in data that is in targets
auto it = std::find_first_of(data.begin(), data.end(),
                             targets.begin(), targets.end());
std::cout << *it; // 8 (first match)
std::cout << std::distance(data.begin(), it); // 3
```

```
// With comparator
auto it2 = std::find_first_of(data.begin(), data.end(),
    targets.begin(), targets.end(),
    [](int a, int b){ return a > b; }); // a > any target
```

SECTION 3 — Transformation & Generation Algorithms

Q14. Explain the two forms of std::transform.

Answer

Unary form: applies a function to each element of one range, writing results to an output range. Binary form: applies a binary function to corresponding pairs of elements from two input ranges. The output range can overlap the input (in-place transform). Both are O(n).

Example

```
std::vector<int> a = {1,2,3,4,5};
std::vector<int> b = {10,20,30,40,50};
std::vector<int> out(5);

// Unary: square each element in-place
std::transform(a.begin(), a.end(), a.begin(),
    [](int x){ return x*x; });
// a: {1,4,9,16,25}

// Binary: element-wise sum
std::transform(a.begin(), a.end(),
               b.begin(), out.begin(),
               std::plus<int>());
// out: {11,24,39,56,75}
```

Q15. What is the difference between std::generate and std::generate_n?

Answer

std::generate assigns the result of a nullary callable to every element in [first, last). std::generate_n assigns results to n elements starting at first and returns an iterator one past the last assigned. Use generate_n with back_inserter to append to a container without pre-sizing.

Example

```
std::vector<int> v(5);

// generate: fill entire pre-sized range
int counter = 0;
std::generate(v.begin(), v.end(), [&]{ return counter++; });
// v: {0,1,2,3,4}

// generate_n: append n elements
std::vector<int> v2;
std::mt19937 rng(42);
std::uniform_int_distribution<int> d(1,100);
std::generate_n(std::back_inserter(v2), 5,
    [&]{ return d(rng); });
// v2 has 5 random ints
```

Q16. What does std::fill and std::fill_n do?

Answer

std::fill assigns a constant value to every element in [first, last). std::fill_n assigns to n elements starting at first and returns an iterator past the last filled element. Both are O(n). Prefer std::fill over a manual loop for clarity; the compiler optimizes them to memset for trivial types.

Example

```
std::vector<int> v(10);

// fill entire range
std::fill(v.begin(), v.end(), 42);
// v: {42,42,42,42,42,42,42,42,42,42}

// fill_n: fill first 5 elements with 0
std::fill_n(v.begin(), 5, 0);
// v: {0,0,0,0,0,42,42,42,42,42}

// fill_n with back_inserter appends
std::vector<int> v2;
std::fill_n(std::back_inserter(v2), 3, 99);
// v2: {99,99,99}
```

Q17. What is std::iota and what problem does it solve?

Answer

std::iota (in <numeric>) fills a range with sequentially incrementing values starting from a given init, using prefix ++. It eliminates manual loops for generating indices or arithmetic sequences. Works with any type supporting ++.

Example

```
#include <numeric>

std::vector<int> v(10);
std::iota(v.begin(), v.end(), 0);
// v: {0,1,2,3,4,5,6,7,8,9}

std::iota(v.begin(), v.end(), 100);
// v: {100,101,...,109}

// Generate indices to sort by value without moving data
std::vector<int> data = {50,10,40,20,30};
std::vector<int> idx(data.size());
std::iota(idx.begin(), idx.end(), 0);
std::sort(idx.begin(), idx.end(),
    [&](int a, int b){ return data[a] < data[b]; });
// idx: {1,3,4,2,0} — sorted indices into data
```

Q18. How does std::replace and std::replace_if work?

Answer

std::replace substitutes all elements equal to old_value with new_value. std::replace_if substitutes all elements satisfying a predicate. replace_copy and replace_copy_if do the same but write to an output range, leaving the input unchanged. All are O(n).

Example

```
std::vector<int> v = {1,2,3,2,4,2,5};

// Replace all 2s with 0
std::replace(v.begin(), v.end(), 2, 0);
// v: {1,0,3,0,4,0,5}

// Replace negatives with 0 (on fresh data)
std::vector<int> v2 = {3,-1,4,-1,5,-9};
std::replace_if(v2.begin(), v2.end(),
    [](int x){ return x < 0; }, 0);
// v2: {3,0,4,0,5,0}

// Non-modifying version
std::vector<int> out;
std::replace_copy(v.begin(), v.end(),
    std::back_inserter(out), 0, 99);
```

Q19. What does std::reverse and std::reverse_copy do?

Answer

std::reverse reverses elements in-place in O(n). std::reverse_copy writes the reversed range to an output iterator, leaving the source unchanged. Both require at least bidirectional iterators. They are equivalent to rotating by 180°.

Example

```
std::vector<int> v = {1,2,3,4,5};

// In-place reverse
std::reverse(v.begin(), v.end());
// v: {5,4,3,2,1}

// Non-destructive reverse
std::vector<int> v2 = {1,2,3,4,5};
std::vector<int> rev;
std::reverse_copy(v2.begin(), v2.end(),
    std::back_inserter(rev));
// v2 unchanged: {1,2,3,4,5}
// rev: {5,4,3,2,1}
```

Q20. What is std::rotate and what does it return?

Answer

std::rotate performs a left rotation: the element at new_first becomes the new first element. All elements before new_first are moved to the end. Returns an iterator to the position where the original first element now lives. Runs in O(n). Used for cyclic shifts and bringing elements to the front.

Example

```
std::vector<int> v = {1,2,3,4,5,6,7};

// Rotate left by 3 (element at index 3 becomes first)
auto it = std::rotate(v.begin(), v.begin()+3, v.end());
// v: {4,5,6,7,1,2,3}
// it points to old first element (1), now at index 3

// Bring minimum to front
auto minIt = std::min_element(v.begin(), v.end());
std::rotate(v.begin(), minIt, v.end());
// Minimum is now v[0]
```

SECTION 4 — Copying, Moving & Partitioning

Q21. What is the difference between std::copy and std::copy_if?

Answer

std::copy copies ALL elements from [first,last) to an output range. std::copy_if copies only elements satisfying a predicate. Both return an iterator one past the last written element. Use back_inserter to append to a container. Both are O(n).

Example

```
std::vector<int> src = {1,2,3,4,5,6,7,8,9,10};
std::vector<int> dst;

// copy all
std::copy(src.begin(), src.end(), std::back_inserter(dst));

// copy_if: only even numbers
std::vector<int> evens;
std::copy_if(src.begin(), src.end(),
    std::back_inserter(evens),
    [](int x){ return x%2 == 0; });
// evens: {2,4,6,8,10}

// copy_n: copy exactly n elements
std::vector<int> first3;
std::copy_n(src.begin(), 3, std::back_inserter(first3));
// first3: {1,2,3}
```

Q22. When is std::copy_backward necessary?

Answer

std::copy_backward copies [first,last) to [result-(last-first), result) copying from last-1 to first. It is essential when source and destination overlap and destination is to the RIGHT of source — forward copy would overwrite elements before they are read. Requires bidirectional iterators.

Example

```
std::vector<int> v = {1,2,3,4,5,6,7};

// Goal: shift elements [0..4] right by 2 (to [2..6])
// Forward copy would corrupt data — use copy_backward
std::copy_backward(v.begin(), v.begin()+5, v.end());
// v: {1,2,1,2,3,4,5}

// Rule: if dst overlaps src and dst > src -> copy_backward
//       if dst overlaps src and dst < src -> copy (forward)
```

Q23. Explain the erase-remove idiom and why remove() alone is insufficient.

Answer

std::remove (and remove_if) shifts "kept" elements to the front and returns a new logical end iterator — it does NOT erase anything or change the container size. Elements beyond the returned iterator are valid but unspecified. To actually shrink the container, chain with container.erase(). This idiom is O(n) total.

Example

```
std::vector<int> v = {1,2,3,4,2,5,2,6};

// After remove, v still has same size!
auto newEnd = std::remove(v.begin(), v.end(), 2);
// v looks like: {1,3,4,5,6,?,?,?} logically
std::cout << v.size(); // Still 8!

// Erase the "garbage" tail
v.erase(newEnd, v.end());
std::cout << v.size(); // Now 5
// v: {1,3,4,5,6}

// One-liner (erase-remove idiom)
v.erase(std::remove_if(v.begin(), v.end(),
    [](int x){ return x%2==0; }), v.end());
```

Q24. What is std::partition, and what does it return?

Answer

std::partition reorders elements so those satisfying a predicate come before those that do not. Returns an iterator to the first element of the second group (the "partition point"). It is NOT stable. Runs in O(n) with O(1) extra memory. The partition point iterator divides the range into the two groups.

Example

```
std::vector<int> v = {3,1,4,1,5,9,2,6,5};

auto pivot = std::partition(v.begin(), v.end(),
    [](int x){ return x < 5; });

// All elements before pivot satisfy x < 5
// All elements from pivot onward do NOT satisfy x < 5
std::cout << std::distance(v.begin(), pivot); // count of < 5

// std::partition_point: binary search for partition boundary
// (requires already-partitioned range)
auto pp = std::partition_point(v.begin(), v.end(),
    [](int x){ return x < 5; });
```

Q25. What is std::stable_partition and when does it matter?

Answer

std::stable_partition is like std::partition but preserves the relative order of elements within each group. With extra memory: O(n). Without (only O(log n) memory available): O(n log n). Important when ordering within groups is meaningful, e.g., partitioning a list of records while maintaining alphabetical order within each group.

Example

```
std::vector<int> v = {3,1,4,1,5,9,2,6,5,3};

// stable_partition: order preserved within groups
std::stable_partition(v.begin(), v.end(),
    [](int x){ return x%2 == 0; }); // evens first
// Evens: {4,2,6} in original relative order
// Odds:  {3,1,1,5,9,5,3} in original relative order
```

```
// vs partition: would give evens first but order not guaranteed
```

Q26. What does std::unique do and how must it be used correctly?

Answer

std::unique removes CONSECUTIVE duplicates (not all duplicates) by overwriting them with subsequent unique values. Returns a past-the-end iterator for the new logical range. The container still holds its original size — you must call erase() to shrink it. For removing ALL duplicates, sort first.

Example

```
std::vector<int> v = {1,1,2,3,3,3,4,4,5};

// unique: removes consecutive duplicates
auto newEnd = std::unique(v.begin(), v.end());
v.erase(newEnd, v.end());
// v: {1,2,3,4,5}

// IMPORTANT: if not sorted first, non-adjacent dups remain
std::vector<int> v2 = {1,3,1,2,3,2};
std::unique(v2.begin(), v2.end()); // Won't remove 1,3 at end!

// Correct: sort then unique
std::sort(v2.begin(), v2.end());
v2.erase(std::unique(v2.begin(), v2.end()), v2.end());
// v2: {1,2,3}
```

SECTION 5 — Numeric Algorithms

Q27. What is std::accumulate and what are its two forms?

Answer

std::accumulate (in <numeric>) computes a running aggregate over a range. Form 1: simple sum with addition. Form 2: custom binary operation (e.g. product, string concatenation). It operates left-to-right in order, starting from init. The result type is the type of init — ensure it is wide enough to avoid overflow.

Example

```
#include <numeric>

std::vector<int> v = {1,2,3,4,5};

// Sum
int sum = std::accumulate(v.begin(), v.end(), 0);
std::cout << sum; // 15

// Product
int prod = std::accumulate(v.begin(), v.end(), 1,
    std::multiplies<int>());
std::cout << prod; // 120

// Concatenate strings
std::vector<std::string> words = {"Hello"," ","World"};
std::string s = std::accumulate(words.begin(), words.end(),
```

```
    std::string(""));
std::cout << s; // "Hello World"
```

Q28. How does std::reduce differ from std::accumulate, and when should you prefer it?

Answer

std::reduce (C++17, in <numeric>) is like accumulate but may execute in any order — enabling parallel execution with std::execution::par. The operation must be associative and commutative for correct results. With seq policy it is functionally equivalent to accumulate. Prefer reduce when parallelism matters and the operation is commutative/associative.

Example

```
#include <numeric>
#include <execution>

std::vector<double> v(1000000, 1.0);

// Sequential: same as accumulate
double sum1 = std::reduce(v.begin(), v.end(), 0.0);

// Parallel: uses thread pool automatically
double sum2 = std::reduce(std::execution::par,
                          v.begin(), v.end(), 0.0);

// Custom op (must be commutative!)
double prod = std::reduce(std::execution::par,
    v.begin(), v.end(), 1.0, std::multiplies<double>());
```

Q29. What is std::inner_product and what is it used for?

Answer

std::inner_product computes the dot product of two ranges: init + (a[0]*b[0]) + (a[1]*b[1]) + ... Custom binary ops for addition and multiplication can be supplied. Runs in O(n). Used in linear algebra, cosine similarity, and weighted sums.

Example

```
#include <numeric>

std::vector<int> a = {1,2,3};
std::vector<int> b = {4,5,6};

// Dot product: 1*4 + 2*5 + 3*6 = 32
int dot = std::inner_product(a.begin(), a.end(),
                             b.begin(), 0);
std::cout << dot; // 32

// Custom ops: sum of squared differences
int ssd = std::inner_product(a.begin(), a.end(),
    b.begin(), 0,
    std::plus<int>(),           // accumulate op
    [](int x, int y){ return (x-y)*(x-y); }); // element op
std::cout << ssd; // (1-4)^2+(2-5)^2+(3-6)^2 = 27
```

Q30. What does std::partial_sum compute?

Answer

std::partial_sum writes prefix sums (or prefix aggregates with custom op) to an output range. Output[i] = input[0] + input[1] + ... + input[i]. The output can alias the input for in-place prefix sums. The inverse of std::adjacent_difference. Runs in O(n). Useful for cumulative distributions and prefix queries.

Example

```
#include <numeric>

std::vector<int> v = {1,2,3,4,5};
std::vector<int> ps(5);

std::partial_sum(v.begin(), v.end(), ps.begin());
// ps: {1,3,6,10,15}  (prefix sums)

// In-place
std::partial_sum(v.begin(), v.end(), v.begin());
// v: {1,3,6,10,15}

// Custom op: prefix product
std::vector<int> v2 = {1,2,3,4,5};
std::partial_sum(v2.begin(), v2.end(), v2.begin(),
    std::multiplies<int>());
// v2: {1,2,6,24,120}
```

Q31. What is std::adjacent_difference and what is it the inverse of?

Answer

std::adjacent_difference writes differences between consecutive elements: out[0]=in[0], out[i]=in[i]-in[i-1]. It is the mathematical inverse of std::partial_sum. With a custom op, it generalizes to any pairwise relationship. Useful for computing deltas, velocities from positions, or rates of change.

Example

```
#include <numeric>

std::vector<int> positions = {0,1,3,6,10,15};
std::vector<int> velocities(6);

// Differences: velocity from positions
std::adjacent_difference(positions.begin(), positions.end(),
                         velocities.begin());
// velocities: {0,1,2,3,4,5}

// Verify inverse: partial_sum of differences -> original
std::vector<int> recovered(6);
std::partial_sum(velocities.begin(), velocities.end(),
                 recovered.begin());
// recovered: {0,1,3,6,10,15} -- original positions restored
```

Q32. What is std::exclusive_scan vs std::inclusive_scan (C++17)?

Answer

inclusive_scan is like partial_sum but supports execution policies for parallel prefix computation. exclusive_scan shifts results by one: out[i] = init + in[0] + ... + in[i-1] (element i is excluded). Both support custom binary ops. They are the parallel-friendly successors to partial_sum.

Example

```
#include <numeric>
```

```
std::vector<int> v = {1,2,3,4,5};
std::vector<int> out(5);

// inclusive_scan: same as partial_sum
std::inclusive_scan(v.begin(), v.end(), out.begin());
// out: {1,3,6,10,15}

// exclusive_scan: shifted by 1 (element i excluded)
std::exclusive_scan(v.begin(), v.end(), out.begin(), 0);
// out: {0,1,3,6,10}

// Parallel version
std::inclusive_scan(std::execution::par,
    v.begin(), v.end(), out.begin());
```

SECTION 6 — Min, Max & Comparison Algorithms

Q33. What is the difference between std::min_element, std::max_element, and std::minmax_element?

Answer

min_element / max_element each make one linear pass returning an iterator to the min or max. minmax_element finds both in a single pass using at most 3n/2 comparisons (more efficient than two separate calls). All return end() for empty ranges. All accept an optional comparator.

Example

```
std::vector<int> v = {3,1,4,1,5,9,2,6,5};

// Separate: 2 full passes, 2*(n-1) comparisons
auto mi = std::min_element(v.begin(), v.end());
auto ma = std::max_element(v.begin(), v.end());

// Together: 1.5 passes, ~3n/2 comparisons
auto [lo, hi] = std::minmax_element(v.begin(), v.end());
std::cout << *lo; // 1
std::cout << *hi; // 9
std::cout << std::distance(v.begin(), lo); // index 1
```

Q34. What does std::clamp do (C++17)?

Answer

std::clamp(v, lo, hi) returns lo if v < lo, hi if v > hi, otherwise returns v. It is equivalent to std::max(lo, std::min(hi, v)) but clearer and requires lo <= hi. Accepts a custom comparator. Useful for bounding values to a valid range.

Example

```
#include <algorithm>

std::cout << std::clamp(5, 0, 10);  // 5  (in range)
std::cout << std::clamp(-3, 0, 10); // 0  (clamped to lo)
std::cout << std::clamp(15, 0, 10); // 10 (clamped to hi)

// Clamp a whole vector
```

```
std::vector<int> v = {-5, 3, 15, 7, -1, 20};
std::transform(v.begin(), v.end(), v.begin(),
    [](int x){ return std::clamp(x, 0, 10); });
// v: {0,3,10,7,0,10}
```

Q35. What does std::lexicographical_compare do?

Answer

std::lexicographical_compare returns true if range [first1,last1) is lexicographically less than [first2,last2) — i.e., it compares element by element and returns at the first difference, or true if the first range is shorter. Used internally by string comparison and sortable tuple operators. Accepts a custom comparator.

Example

```
std::vector<int> a = {1,2,3};
std::vector<int> b = {1,2,4};
std::vector<int> c = {1,2};

std::cout << std::lexicographical_compare(
    a.begin(),a.end(), b.begin(),b.end()); // 1 (a < b: 3<4)

std::cout << std::lexicographical_compare(
    c.begin(),c.end(), a.begin(),a.end()); // 1 (c shorter)

// Case-insensitive string comparison
std::string s1="Hello", s2="hello";
bool less = std::lexicographical_compare(
    s1.begin(),s1.end(), s2.begin(),s2.end(),
    [](char a,char b){return tolower(a)<tolower(b);});
```

Q36. What does std::equal do and how does it handle ranges of different lengths?

Answer

std::equal returns true if two ranges have identical elements. The two-range form (C++14) takes both begin+end for the second range and handles different lengths safely. The three-iterator form assumes the second range is at least as long as the first — UB if shorter. An optional predicate generalizes equality.

Example

```
std::vector<int> a = {1,2,3,4,5};
std::vector<int> b = {1,2,3,4,5};
std::vector<int> c = {1,2,3};

std::cout << std::equal(a.begin(),a.end(),
                        b.begin(),b.end()); // 1
std::cout << std::equal(a.begin(),a.end(),
                        c.begin(),c.end()); // 0 (diff size)

// Custom predicate: compare absolute values
std::vector<int> d = {1,-2,3,-4,5};
std::cout << std::equal(a.begin(),a.end(),d.begin(),d.end(),
    [](int x,int y){ return std::abs(x)==std::abs(y); }); // 1
```

Q37. What is std::mismatch and what does it return?

Answer

std::mismatch compares two ranges element by element and returns a pair of iterators to the FIRST position where they differ. If equal throughout, returns {end1, end2} (or {end1, corresponding position in range2}). Useful for string diff, prefix comparison, and change detection.

Example

```
std::string s1 = "Hello World";
std::string s2 = "Hello Earth";

auto [i1, i2] = std::mismatch(s1.begin(), s1.end(),
                              s2.begin());

std::cout << std::distance(s1.begin(), i1); // 6
std::cout << *i1; // W
std::cout << *i2; // E

// Common prefix length
size_t prefixLen = std::distance(s1.begin(), i1); // 6
```

SECTION 7 — Heap Algorithms

Q38. Explain the four heap algorithms: make_heap, push_heap, pop_heap, sort_heap.

Answer

make_heap converts a range into a max-heap in O(n). push_heap assumes all but the last element is a valid heap and sifts the last element up: O(log n) — call push_back first. pop_heap moves the max to the last position and restores heap property for [first, last-1): O(log n) — call pop_back to remove. sort_heap sorts a heap in O(n log n).

Example

```
std::vector<int> v = {5,3,8,1,9,2};

// Build heap O(n)
std::make_heap(v.begin(), v.end());
std::cout << v[0]; // 9 (max)

// Add element
v.push_back(15);
std::push_heap(v.begin(), v.end()); // O(log n)
std::cout << v[0]; // 15

// Remove max
std::pop_heap(v.begin(), v.end()); // Max -> back
std::cout << v.back(); // 15
v.pop_back();

// Full heap sort
std::sort_heap(v.begin(), v.end()); // O(n log n)
```

Q39. How does std::is_heap and std::is_heap_until work?

Answer

std::is_heap returns true if a range satisfies the max-heap property (every parent >= its children). std::is_heap_until returns an iterator to the first position where the heap property is violated (returns end() if the whole range is a valid heap). Both run in O(n). Useful for debugging heap-based algorithms.

Example

```
std::vector<int> v = {9,5,8,1,3,2,7};

std::cout << std::is_heap(v.begin(), v.end()); // 1 (valid heap)

// Break the heap property
v[1] = 10; // Child (10) > parent (9) -- invalid!
std::cout << std::is_heap(v.begin(), v.end()); // 0

auto it = std::is_heap_until(v.begin(), v.end());
std::cout << std::distance(v.begin(), it); // 1 (breaks at index 1)
```

SECTION 8 — Permutation & Set Algorithms

Q40. How does std::next_permutation work and what does it return?

Answer

std::next_permutation rearranges [first, last) to the next lexicographically greater permutation. Returns true if successful; returns false and resets to the smallest permutation (sorted ascending) when already at the last permutation. Iterating with do-while over next_permutation enumerates all n! permutations in order.

Example

```
std::vector<int> v = {1,2,3};

// Enumerate all 6 permutations
do {
    for (int x : v) std::cout << x;
    std::cout << " ";
} while(std::next_permutation(v.begin(), v.end()));
// 123 132 213 231 312 321

// NOTE: start from sorted for complete enumeration
// std::prev_permutation goes the other direction
std::sort(v.begin(), v.end(), std::greater<int>());// {3,2,1}
std::prev_permutation(v.begin(), v.end()); // {3,1,2}
```

Q41. What do the set algorithms (set_union, set_intersection, set_difference, set_symmetric_difference) require?

Answer

All four set algorithms require SORTED input ranges. They produce sorted output in O(n+m). set_union: elements in either; set_intersection: elements in both; set_difference: in first but not second; set_symmetric_difference: in either but not both. They work on any sorted range, not just std::set.

Example

```
std::vector<int> a = {1,2,3,4,5};
std::vector<int> b = {3,4,5,6,7};
std::vector<int> out;
```

```
std::set_union(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(out));       // {1,2,3,4,5,6,7}
out.clear();

std::set_intersection(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(out));       // {3,4,5}
out.clear();

std::set_difference(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(out));       // {1,2}
out.clear();

std::set_symmetric_difference(a.begin(),a.end(),
    b.begin(),b.end(),std::back_inserter(out)); // {1,2,6,7}
```

Q42. What does std::includes do?

Answer

std::includes(first1,last1,first2,last2) returns true if every element of the second sorted range appears in the first sorted range. It is O(n+m). Useful for checking if one set is a subset of another without creating intermediate containers.

Example

```
std::vector<int> superset = {1,2,3,4,5,6,7,8,9};
std::vector<int> subset1  = {2,4,6};
std::vector<int> subset2  = {2,4,11};

std::cout << std::includes(
    superset.begin(), superset.end(),
    subset1.begin(),  subset1.end()); // 1 (2,4,6 all present)

std::cout << std::includes(
    superset.begin(), superset.end(),
    subset2.begin(),  subset2.end()); // 0 (11 not present)
```

Q43. What does std::is_permutation do and how does it differ from sorting+comparing?

Answer

std::is_permutation(first1,last1,first2,last2) returns true if the two ranges contain the same elements in any order. It does NOT require sorting and does NOT modify the ranges. Complexity: O(n²) in the general case (O(n) if elements are sortable and you use sort+equal instead). Use when you cannot sort, or for small ranges.

Example

```
std::vector<int> a = {1,2,3,4,5};
std::vector<int> b = {5,3,1,4,2};
std::vector<int> c = {1,2,3,4,6};

std::cout << std::is_permutation(
    a.begin(),a.end(),b.begin(),b.end()); // 1 (same elements)

std::cout << std::is_permutation(
    a.begin(),a.end(),c.begin(),c.end()); // 0 (6 differs)

// Faster alternative when sorting is OK:
std::sort(a.begin(),a.end());
std::sort(b.begin(),b.end());
bool perm = (a == b); // O(n log n) vs O(n^2)
```

SECTION 9 — Merge, Scan & Advanced Algorithms

Q44. What is std::merge and when should you use std::inplace_merge instead?

Answer

std::merge takes two separate sorted ranges and writes the merged output to a third range — O(n+m), requires O(n+m) output space. std::inplace_merge merges two adjacent sorted sub-ranges within the same container in-place — O(n log n) without extra memory, O(n) with. Use merge for separate containers; inplace_merge for merge-sort or sorted range fusion.

Example

```
std::vector<int> a = {1,3,5,7};
std::vector<int> b = {2,4,6,8};
std::vector<int> merged;

// std::merge: two separate ranges -> output
std::merge(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(merged));
// merged: {1,2,3,4,5,6,7,8}

// std::inplace_merge: adjacent halves of one vector
std::vector<int> v = {1,3,5,7, 2,4,6,8};
std::inplace_merge(v.begin(), v.begin()+4, v.end());
// v: {1,2,3,4,5,6,7,8}
```

Q45. What does std::for_each return and when is that useful?

Answer

std::for_each returns the function object passed to it (after applying it to all elements). This is useful for stateful function objects — the returned functor contains accumulated state from processing all elements. Lambdas are commonly used but the return makes stateful functors extra powerful.

Example

```
struct Stats {
    int count = 0;
    double sum = 0;
    void operator()(int x) { ++count; sum += x; }
    double mean() const { return sum/count; }
};

std::vector<int> v = {1,2,3,4,5,6,7,8,9,10};

// for_each RETURNS the functor with accumulated state
Stats stats = std::for_each(v.begin(), v.end(), Stats{});
std::cout << stats.mean(); // 5.5
std::cout << stats.count;  // 10
```

Q46. Explain the three quantifier algorithms: all_of, any_of, none_of.

Answer

All three test a predicate over a range and short-circuit on the first decisive element. all_of: true if predicate holds for every element (empty range -> true). any_of: true if predicate holds for at least one element (empty range -> false). none_of: true if predicate holds for no element (empty range -> true).

Example

```
std::vector<int> v = {2,4,6,8,10};
auto isEven = [](int x){ return x%2 == 0; };

std::cout << std::all_of(v.begin(),v.end(),isEven);  // 1
std::cout << std::any_of(v.begin(),v.end(),
    [](int x){return x>8;}); // 1 (10>8)
std::cout << std::none_of(v.begin(),v.end(),
    [](int x){return x<0;}); // 1

// Short-circuit: stops at first failure
std::vector<int> v2 = {2,4,7,8,10};
std::cout << std::all_of(v2.begin(),v2.end(),isEven); // 0 (stops at 7)
```

Q47. What does std::count and std::count_if do?

Answer

std::count counts occurrences of a specific value in O(n). std::count_if counts elements satisfying a predicate in O(n). For sorted containers, prefer lower_bound/upper_bound for O(log n) counting, or the container's own count() member (map, set, multimap, etc.).

Example

```
std::vector<int> v = {1,2,3,2,1,2,4,2,5};

std::cout << std::count(v.begin(), v.end(), 2);    // 4

std::cout << std::count_if(v.begin(), v.end(),
    [](int x){ return x%2==0; }); // 4 (2,2,2,4 -> wait: 4 evens)

// For multiset/multimap use member count() — O(log n + k)
std::multiset<int> ms(v.begin(), v.end());
std::cout << ms.count(2); // 4
```

Q48. What is std::sample (C++17) and how does it guarantee unbiased sampling?

Answer

std::sample(first, last, out, n, gen) selects n elements uniformly at random without replacement from the range. It implements reservoir sampling, which gives each element equal probability of being selected regardless of range size. Unlike shuffle+take, it works on forward iterators (not random access required for the source).

Example

```
#include <algorithm>
#include <random>

std::vector<int> population(100);
std::iota(population.begin(), population.end(), 1); // 1..100

std::mt19937 rng(std::random_device{}());

// Sample 10 elements uniformly without replacement
std::vector<int> sample(10);
std::sample(population.begin(), population.end(),
            sample.begin(), 10, rng);
```

```
// Output is in original order (not shuffled)
for (int x : sample) std::cout << x << " ";
```

Q49. What does std::shuffle do and why is it preferred over std::random_shuffle?

Answer

std::shuffle(first, last, gen) randomly reorders all elements using a UniformRandomBitGenerator. std::random_shuffle (deprecated C++14, removed C++17) used rand() which has poor statistical properties and is not seedable per-algorithm. Always use std::shuffle with std::mt19937 for correct, reproducible randomness.

Example

```
#include <algorithm>
#include <random>

std::vector<int> v = {1,2,3,4,5,6,7,8,9,10};

// Correct approach: mt19937 + shuffle
std::mt19937 rng(42); // Seed for reproducibility
std::shuffle(v.begin(), v.end(), rng);

// BAD (deprecated/removed):
// std::random_shuffle(v.begin(), v.end()); // DO NOT USE

// Verify all elements still present
std::sort(v.begin(), v.end());
// v: {1,2,3,...,10}
```

Q50. What does std::rotate_copy do?

Answer

std::rotate_copy writes a rotated copy of a range to an output iterator, leaving the original range unchanged. Equivalent to calling std::rotate on a copy. Returns an iterator past the last written element. O(n).

Example

```
std::vector<int> v = {1,2,3,4,5,6,7};
std::vector<int> out;

// Write rotated copy (pivot at index 3)
std::rotate_copy(v.begin(), v.begin()+3, v.end(),
    std::back_inserter(out));
// out: {4,5,6,7,1,2,3}
// v:   {1,2,3,4,5,6,7} -- unchanged
```

SECTION 10 — Parallel Algorithms & C++17/20 Additions

Q51. What are execution policies in C++17 and what are the three standard policies?

Answer

Execution policies are passed as the first argument to most STL algorithms to request a specific execution strategy. std::execution::seq: sequential (default behavior). std::execution::par: parallel using a thread pool. std::execution::par_unseq: parallel + vectorized (SIMD). Predicates/callables must be thread-safe with par and par_unseq.

Example

```
#include <execution>
#include <algorithm>

std::vector<int> v(10000000);
std::iota(v.begin(), v.end(), 0);

// Sequential
std::sort(std::execution::seq,  v.begin(), v.end());

// Parallel (multi-threaded)
std::sort(std::execution::par,  v.begin(), v.end());

// Parallel + SIMD
std::for_each(std::execution::par_unseq,
    v.begin(), v.end(), [](int& x){ x *= 2; });
```

Q52. What is std::transform_reduce (C++17) and how does it combine algorithms?

Answer

std::transform_reduce applies a transformation to elements (or pairs from two ranges) then reduces the results. It is the parallel-friendly MapReduce in one call. Replaces inner_product for parallel workloads. The reduction must be associative and commutative for parallel execution.

Example

```
#include <numeric>
#include <execution>

std::vector<int> a = {1,2,3,4,5};
std::vector<int> b = {10,20,30,40,50};

// Dot product (map=multiply, reduce=add)
int dot = std::transform_reduce(
    a.begin(), a.end(), b.begin(), 0);
std::cout << dot; // 550

// Parallel sum of squares
long long ssq = std::transform_reduce(
    std::execution::par,
    a.begin(), a.end(), 0LL,
    std::plus<long long>(),
    [](int x){ return (long long)x*x; });
std::cout << ssq; // 55
```

Q53. What are the C++20 ranges versions of algorithms and what advantage do projections give?

Answer

std::ranges:: versions of algorithms (ranges::sort, ranges::find, etc.) take ranges instead of iterator pairs, support projections (transform elements before comparing), and give cleaner error messages via concepts. A projection is a callable applied to each element before the comparator — it avoids creating a custom comparator just to compare a member.

Example

```
#include <algorithm>
#include <ranges>

struct Person { std::string name; int age; };
std::vector<Person> people = {
    {"Alice",30}, {"Bob",25}, {"Carol",35}
};

// Sort by age using projection -- no custom lambda needed!
std::ranges::sort(people, {}, &Person::age);
// Bob(25), Alice(30), Carol(35)

// Find by name projection
auto it = std::ranges::find(people, "Alice", &Person::name);
std::cout << it->age; // 30

// Min by age with projection
auto youngest = std::ranges::min_element(people, {}, &Person::age);
```

Q54. What is std::transform_exclusive_scan and std::transform_inclusive_scan (C++17)?

Answer

These combine a unary transform with a prefix scan (exclusive or inclusive). transform_inclusive_scan(first,last,out,op,transform): applies transform to each element then computes prefix scan. transform_exclusive_scan adds an init value shifted by one. Both support parallel execution policies.

Example

```
#include <numeric>

std::vector<int> v = {1,2,3,4,5};
std::vector<int> out(5);

// Square each element, then prefix sum
std::transform_inclusive_scan(v.begin(), v.end(),
    out.begin(),
    std::plus<int>(),        // scan op
    [](int x){return x*x;}); // transform
// Squares: 1,4,9,16,25
// Prefix:  {1,5,14,30,55}

// Exclusive version (shifted by 1 with init=0)
std::transform_exclusive_scan(v.begin(), v.end(),
    out.begin(), 0,
    std::plus<int>(), [](int x){return x*x;});
// out: {0,1,5,14,30}
```

Q55. What are constexpr algorithms in C++20?

Answer

In C++20, most standard algorithms (sort, find, fill, copy, etc.) are marked constexpr, enabling them to run at compile time in constant expressions. This allows compile-time sorting of lookup tables, compile-time validation, and consteval functions that use STL algorithms without runtime overhead.

Example

```
constexpr std::array<int,5> makeSorted() {
    std::array<int,5> arr = {5,2,8,1,9};
    std::sort(arr.begin(), arr.end()); // constexpr sort!
```

```
    return arr;
}

constexpr auto sorted = makeSorted();
// sorted = {1,2,5,8,9} at compile time!

static_assert(sorted[0] == 1);
static_assert(sorted[4] == 9);

// Compile-time binary search
constexpr bool found = std::binary_search(
    sorted.begin(), sorted.end(), 5); // true at compile time
```

SECTION 11 — Iterator Utilities & Adapter Algorithms

Q56. What does std::advance, std::next, and std::prev do?

▶ **Answer**

std::advance(it, n) moves an iterator by n positions in-place (O(1) for random access, O(n) for forward/bidirectional). std::next(it, n=1) returns a new iterator advanced by n (non-mutating). std::prev(it, n=1) returns a new iterator moved back by n. Prefer next/prev over advance for clarity and because they do not modify the original iterator.

⌨ **Example**

```
std::list<int> lst = {1,2,3,4,5};
auto it = lst.begin();

// advance: modifies in place
std::advance(it, 3);
std::cout << *it; // 4

// next: returns new, does not modify it
auto it2 = std::next(lst.begin(), 2);
std::cout << *it2; // 3

// prev: goes backward
auto it3 = std::prev(lst.end(), 2);
std::cout << *it3; // 4

// For random-access: all are O(1)
// For list/bidirectional: O(n) but correct
```

Q57. What does std::distance do and what is its complexity?

▶ **Answer**

std::distance(first, last) returns the number of increments from first to last. For random-access iterators (vector, deque, array) it is O(1) via pointer arithmetic. For other iterators (list, set, etc.) it is O(n) by incrementing. Never call distance in a hot loop on non-random-access iterators.

⌨ **Example**

```
std::vector<int> v = {1,2,3,4,5};
std::list<int>   l = {1,2,3,4,5};
```

```
// O(1) for random-access
auto dist_v = std::distance(v.begin(), v.end()); // 5

// O(n) for bidirectional — traverses the list!
auto dist_l = std::distance(l.begin(), l.end()); // 5

// Practical use: index of found element
auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end())
    std::cout << std::distance(v.begin(), it); // 2
```

Q58. What is std::iter_swap and when is it preferred over std::swap?

Answer

std::iter_swap(it1, it2) swaps the VALUES pointed to by two iterators, regardless of whether the iterators themselves can be swapped. It is equivalent to std::swap(*it1, *it2) but is the canonical form for algorithms operating generically over iterators. Algorithms like sort and partition use iter_swap internally.

Example

```
std::vector<int> v = {1,2,3,4,5};

// Swap values at indices 0 and 4
std::iter_swap(v.begin(), v.begin()+4);
// v: {5,2,3,4,1}

// Equivalent to:
std::swap(v[0], v[4]);

// Generic algorithm using iter_swap
template<typename It>
void mySwap(It a, It b) {
    std::iter_swap(a, b); // Works for any iterator type
}
```

Q59. What is std::copy_n and how does it differ from std::copy?

Answer

std::copy_n(first, n, out) copies exactly n elements starting from first to out. It does not need a last iterator — useful when you know the count but not the end iterator (e.g., C arrays without end pointer). Returns an iterator past the last written element. O(n).

Example

```
int c_arr[] = {10,20,30,40,50,60};
std::vector<int> dst;

// copy_n: copy 4 from C array (no end pointer needed)
std::copy_n(c_arr, 4, std::back_inserter(dst));
// dst: {10,20,30,40}

// Practical: copy stream data in chunks
std::vector<int> src(100);
std::iota(src.begin(), src.end(), 1);

std::vector<int> chunk(10);
auto next_start = std::copy_n(src.begin(), 10, chunk.begin());
// chunk: first 10, next_start points to src[10]
```

Q60. What are the C++20 std::ranges utility functions: ranges::distance, ranges::advance, ranges::next, ranges::prev?

▶️ Answer

C++20 adds ranges-aware versions of iterator utilities. ranges::distance takes a range directly (no need to pass begin+end separately). ranges::advance, next, prev are constrained by concepts and work with sentinels (non-iterator end markers), enabling support for null-terminated strings, FILE* ranges, etc. They are composable with ranges views.

⌨️ Example

```
#include <ranges>
#include <iterator>

std::vector<int> v = {1,2,3,4,5};

// C++20: take range directly
std::cout << std::ranges::distance(v); // 5

// Works with view pipelines
auto filtered = v | std::views::filter([](int x){return x>2;});
std::cout << std::ranges::distance(filtered); // 3

// ranges::next with sentinel support
auto it = std::ranges::next(v.begin(), 2);
std::cout << *it; // 3

// Bounded advance (stops at sentinel)
std::ranges::advance(it, 100, v.end()); // stops at end()
```